



100 MANDATORY JAVASCRIPT INTERVIEW QUESTIONS



TECH
TELLIPTICA

What are the different ways to include JavaScript in HTML?

Answer: There are three ways:

1. **Inline JavaScript:** Using `<script>` tag directly in HTML

```
<script>
console.log("Hello World");
</script>
```

2. **External JavaScript:** Linking an external `.js` file

```
<script src="script.js"></script>
```

3. **Inline event handlers:** Using JavaScript directly in HTML attributes

```
<button onclick="alert('Clicked')">Click me</button>
```

What is the difference between `==` and `===`?

Answer:

- `==` (loose equality) performs type coercion before comparison
- `===` (strict equality) checks both value and type without coercion

```
5 == "5"    // true (type coercion)
5 === "5"   // false (different types)
null == undefined // true
null === undefined // false
```

What is the difference between `null` and `undefined`?

Answer:

- `undefined`: Variable declared but not assigned a value, or function parameter not provided
- `null`: Intentional absence of value, explicitly assigned

```
let x;  
console.log(x);           // undefined  
let y = null;  
console.log(y);           // null  
console.log(typeof x);    // "undefined"  
console.log(typeof y);    // "object" (known bug in JS)
```

What is NaN and how to check for it?

Answer: NaN stands for "Not a Number" and represents a value that is not a valid number.

```
console.log(typeof NaN);           // "number"  
console.log(NaN === NaN);          // false  
console.log(Number.isNaN(NaN));    // true (recommended)  
console.log(isNaN("hello"));       // true  
console.log(Number.isNaN("hello")); // false (more accurate)
```

What is the difference between `let`, `const`, and `var`?

Answer:

- `var`: Function-scoped, hoisted, can be redeclared
- `let`: Block-scoped, hoisted but in temporal dead zone, cannot be redeclared
- `const`: Block-scoped, must be initialized, cannot be reassigned (but objects/arrays can be mutated)

```
// var - function scoped
function test() {
  if (true) {
    var x = 1;
  }
  console.log(x); // 1 (accessible)
}

// let - block scoped
if (true) {
  let y = 1;
}
console.log(y); // ReferenceError

// const - cannot reassign
const z = 1;
z = 2; // TypeError
```

What is type coercion in JavaScript?

Answer: Type coercion is the automatic conversion of values from one data type to another.

```
// Implicit coercion
"5" + 3 "5" // "53" (number to string)
- 3 true + // 2 (string to number)
1 "" + // 2 (boolean to number)
false // "false" (boolean to string)

// Explicit coercion
Number("123") // 123
String(123) // "123"
Boolean(1) // true
```

What are truthy and falsy values?

Answer:

- **Falsy values:** `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`
- **Truthy values:** Everything else (including `[]`, `{}`, `"0"`, `"false"`)

```
if (0) console.log("won't print");  
if ([]) console.log("will print"); // empty array is truthy  
if ({}) console.log("will print"); // empty object is truthy
```

What is the difference between `null` and `undefined` in terms of type checking?

Answer:

```
typeof null // "object" (known bug)  
typeof undefined // "undefined"  
null == undefined // true (loose equality)  
null === undefined // false (strict equality)
```

What is the use of `use strict`?

Answer: `"use strict"` enables strict mode, which catches common coding errors and prevents unsafe actions.

```
"use strict";  
x = 3.14; // Error: x is not defined  
delete Object.prototype; // Error: cannot delete  
function duplicate(a, a) { } // Error: duplicate parameter
```

Variables & Data Types

What are the primitive data types in JavaScript?

Answer: JavaScript has 7 primitive types:

1. `Number` - integers and floating-point numbers

2. `String` - text data
3. `Boolean` - true or false
4. `Undefined` - uninitialized variable
5. `Null` - intentional absence of value
6. `Symbol` - unique identifier (ES6)
7. `BigInt` - large integers (ES2020)

What is the difference between primitive and reference types?

Answer:

- **Primitive types:** Stored directly in variable, immutable, compared by value
- **Reference types:** Stored as reference, mutable, compared by reference

```
// Primitive let a = 5; let b =  
a; b = 10; console.log(a); // 5  
(unchanged)  
  
// Reference  
let obj1 = {x: 5};  
let obj2 = obj1;  
obj2.x = 10;  
console.log(obj1.x); // 10 (changed)
```

How do you check if a variable is an array?

Answer:

```
const arr = [1, 2, 3];  
Array.isArray(arr);           // true (recommended)  
arr instanceof Array;         // true  
Object.prototype.toString.call(arr) === '[object Array]'; // true
```

What is the difference between `slice()` and `splice()` ?

Answer:

- `slice()` : Returns a new array, doesn't modify original
- `splice()` : Modifies the original array, returns removed elements

```
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3);    // [2, 3] (new array)
console.log(arr);   // [1, 2, 3, 4, 5] (unchanged)

arr.splice(1, 3);   // [2, 3, 4] (removed elements)
console.log(arr);   // [1, 5] (modified)
```

How do you convert a string to a number?

Answer:

```
Number("123");      // 123
parseInt("123");     // 123
parseFloat("3.14"); // 3.14
+"123";             // 123 (unary plus)
"123" * 1;          // 123
```

What is the difference between `parseInt()` and `Number()` ?

Answer:

```
parseInt("123abc"); // 123 (stops at first non-numeric)
Number("123abc");   // NaN (requires entire string to be numeric)
parseInt("10", 2);  // 2 (can specify radix)
Number("10", 2);    // 10 (ignores radix)
```

What is template literal?

Answer: Template literals (backticks) allow embedded expressions and multi-line strings.

```
const name = "John"; const age = 30; const message = `Hello,
${name}! You are ${age} years old.`; const multiLine = `
Line 1   Line 2 `;
```

What is the difference between `String()` and `.toString()` ?
Answer:

```
String(null);           // "null"
String(undefined);      // "undefined"
null.toString();        // TypeError
undefined.toString();   // TypeError
```

How do you check if a number is an integer?
Answer:

```
Number.isInteger(5);      // true
Number.isInteger(5.0);    // true
Number.isInteger(5.5);    // false
Number.isInteger("5");    // false
```

What is BigInt and when to use it?

Answer: BigInt represents integers larger than `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$).

```
const bigNum = 9007199254740991n;
const bigNum2 = BigInt("9007199254740991");
console.log(bigNum + 1n); // 9007199254740992n
// Cannot mix BigInt with Number without conversion
```


Functions

What is a function declaration vs function expression?

Answer:

```
// Function declaration (hoisted)
function greet() {
  return "Hello";
}

// Function expression (not hoisted)
const greet = function() {
  return "Hello";
};

// Arrow function expression
const greet = () => "Hello";
```

What is hoisting?

Answer: Hoisting is JavaScript's behavior of moving declarations to the top of their scope.

```
console.log(x); // undefined (not ReferenceError)
var x = 5;

// Function declarations are fully hoisted
sayHello(); // "Hello"
function sayHello() {
  console.log("Hello");
}
```

What is a closure?

Answer: A closure is a function that has access to variables in its outer scope even after the outer function has returned.

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  }; } const counter =  
outer();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

What is the difference between arrow functions and regular functions?

Answer:

1. **this binding**: Arrow functions use lexical this, regular functions use dynamic `this`
2. **Arguments object**: Arrow functions don't have `arguments`
3. **Constructor**: Arrow functions can't be used as constructors
4. **Hoisting**: Arrow functions are not hoisted

```
const obj = {  
  name: "John",  
  regular: function() {  
    return this.name; // "John"  
  },  
  arrow: () => {  
    return this.name; // undefined (lexical this)  
  }  
};
```

What are default parameters?

Answer: Default parameters allow function parameters to have default values.

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}  
greet();           // "Hello, Guest"  
greet("John");    // "Hello, John"
```

What are rest parameters?

Answer: Rest parameters allow a function to accept an indefinite number of arguments as an array.

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
sum(1, 2, 3, 4); // 10
```

What is the difference between `call()`, `apply()`, and `bind()` ?

Answer:

- `call()`: Invokes function with specified `this` and arguments (comma-separated)
- `apply()`: Same as `call()` but arguments as array
- `bind()`: Returns new function with bound `this`

```
const obj = { name: "John" };  
function greet(greeting, punctuation) {  
  return `${greeting}, ${this.name}${punctuation}`;  
}  
  
greet.call(obj, "Hello", "!");           // "Hello, John!"  
greet.apply(obj, ["Hello", "!"]);        // "Hello, John!"  
const bound = greet.bind(obj);  
bound("Hello", "!");                     // "Hello, John!"
```

What is an IIFE (Immediately Invoked Function Expression)?

Answer: IIFE is a function that runs as soon as it's defined.

```
(function() {  
  console.log("IIFE executed");  
})();  
  
// With parameters  
(function(name) {  
  console.log("Hello", name);  
})("John");
```

What is a higher-order function?

Answer: A function that takes other functions as arguments or returns a function.

```
// Function as argument  
function operate(a, b, operation) {  
  return operation(a, b);  
}  
operate(5, 3, (x, y) => x + y); // 8  
  
// Function as return value  
function multiplier(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}  
const double = multiplier(2);  
double(5); // 10
```

What is recursion?

Answer: Recursion is when a function calls itself.

```
function factorial(n) {  if (n
<= 1) return 1;  return n *
factorial(n      -      1);  }
factorial(5); // 120
```

Objects & Arrays

How do you create an object in JavaScript?

Answer:

```
// Object literal
const obj1 = { name: "John", age: 30 };

// Constructor function
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const obj2 = new Person("John", 30);

// Object.create()
const obj3 = Object.create({});
obj3.name = "John";

// Class (ES6) class
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}
const obj4 = new Person("John",
30);
```

What is object destructuring?

Answer: Destructuring allows extracting values from objects into variables.

```
const person = { name: "John", age: 30, city: "NYC" };  
const { name, age } = person; const { name: fullName,  
age: years } = person; const { name, age, country =  
"USA" } = person;
```

What is array destructuring?

Answer:

```
const arr = [1, 2, 3]; const [a, b, c] = arr;  
const [first, ...rest] = arr; const [,  
second] = arr; // skip first element
```

What is the spread operator?

Answer: Spread operator (`...`) expands arrays/objects into individual elements.

```
// Arrays  
const arr1 = [1, 2, 3];  
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]  
  
// Objects  
const obj1 = { a: 1, b: 2 };  
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }  
  
// Function arguments  
Math.max(...[1, 2, 3]); // 3
```

What is the difference between `map()`, `filter()`, and `reduce()` ?

Answer:

- `map()` : Transforms each element, returns new array

- `filter()` : Filters elements based on condition, returns new array
- `reduce()` : Reduces array to single value

```
const numbers = [1, 2, 3, 4, 5];
numbers.map(n => n * 2);           // [2, 4, 6, 8, 10]
numbers.filter(n => n % 2 === 0); // [2, 4]
numbers.reduce((a, b) => a + b, 0); // 15
```

What is the difference between `forEach()` and `map()` ?

Answer:

- `forEach()` : Executes function for each element, returns `undefined`
- `map()` : Executes function for each element, returns new array

```
const arr = [1, 2, 3];
arr.forEach(n => console.log(n)); // logs each, returns undefined
arr.map(n => n * 2);               // returns [2, 4, 6]
```

How do you check if an object has a property?

Answer:

```
const obj = { name: "John", age: 30 };
"name" in obj; // true
obj.hasOwnProperty("name"); // true
Object.hasOwn(obj, "name"); // true (ES2022)
obj.name !== undefined;    // true (but checks prototype too)
```

What is the difference between `Object.keys()`, `Object.values()`, and `Object.entries()` ?

Answer:

```
const obj = { a: 1, b: 2, c: 3 };
Object.keys(obj);    // ["a", "b", "c"]
Object.values(obj);  // [1, 2, 3]
Object.entries(obj); // [["a", 1], ["b", 2], ["c", 3]]
```

How do you deep copy an object?

Answer:

```
const obj = { a: 1, b: { c: 2 } };

// Shallow copy (not deep)
const shallow = { ...obj };

// Deep copy methods
const deep1 = JSON.parse(JSON.stringify(obj)); // Simple but has limita
const deep2 = structuredClone(obj); // ES2022, better
```

What is the difference between `push()` and `unshift()` ?

Answer:

- `push()` : Adds elements to the end of array
- `unshift()` : Adds elements to the beginning of array

```
const arr = [2, 3];
arr.push(4);    // [2, 3, 4]
arr.unshift(1); // [1, 2, 3, 4]
```


Scope & Hoisting

What is scope in JavaScript?

Answer: Scope determines the accessibility of variables, functions, and objects in different parts of code.

```
// Global scope
var global = "global";

function test() {
  // Function scope
  var functionScoped = "function";
  if (true) {
    // Block scope (let/const)
    let blockScoped = "block";
  }
}
```

What is the difference between function scope and block scope?

Answer:

```
// Function scope (var)
function test() {
  if (true) {
    var x = 1;
  }
  console.log(x); // 1 (accessible)
}

// Block scope (let/const)
function test() {
  if (true) {
    let y = 1;
  }
  console.log(y); // ReferenceError
}
```

What is the temporal dead zone?

Answer: The period between entering scope and declaration where variables cannot be accessed.

```
console.log(x); // ReferenceError (TDZ)
let x = 5;
```

What is lexical scope?

Answer: Lexical scope means variables are accessible based on where they're defined in the source code.

```
function outer() {
  let x = "outer";
  function inner() {
    console.log(x); // "outer" (lexical scope)
  }
  inner();
}
```

What is the global scope?

Answer: Variables declared outside functions are in global scope and accessible everywhere.

```
var globalVar = "global";
function test() {
  console.log(globalVar); // "global"
}
```

ES6+ Features

What are arrow functions?

Answer: Arrow functions provide shorter syntax and lexical `this` binding.

```
// Regular function
const add = function(a, b) {
  return a + b;
};

// Arrow function
const add = (a, b) => a + b;
const greet = name => `Hello, ${name}`;
const noParams = () => "Hello";
```

What are template literals?

Answer: Template literals use backticks for string interpolation and multi-line strings.

```
const name = "John"; const message
= `Hello,    ${name}!`;    const
multiLine = `  Line 1  Line 2 `;
```

What are classes in JavaScript?

Answer: Classes are syntactic sugar over prototype-based inheritance.

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    return `Hello, I'm ${this.name}`;  
  }  
  
  static getSpecies() {  
    return "Homo sapiens";  
  }  
}
```

What is inheritance in JavaScript?

Answer:

```
class Student extends Person {  
  constructor(name, age, grade) {  
    super(name, age);  
    this.grade = grade;  
  }  
  
  study() {  
    return `${this.name} is studying`;  
  }  
}
```

What are modules?

Answer: Modules allow code organization and reusability.

```
// math.js
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}

// app.js
import { PI, add } from './math.js';
```

What are Sets and Maps?

Answer:

```
// Set - unique values
const set = new Set([1, 2, 3, 3, 4]); // {1, 2, 3, 4}
set.add(5);
set.has(3); // true

// Map - key-value pairs
const map = new Map();
map.set("key", "value");
map.get("key"); // "value"
```

What are Symbols?

Answer: Symbols are unique, immutable primitive values.

```
const sym1 = Symbol("description");
const sym2 = Symbol("description");
sym1 === sym2; // false (always unique)

const obj = {
  [sym1]: "value"
};
```

What is optional chaining (?.)?

Answer: Optional chaining safely accesses nested object properties.

```
const user = { profile: { name: "John" } };
user.profile?.name;           // "John"
user.contact?.phone;          // undefined (no error)
user.method?.();               // undefined (safe method call)
```

What is nullish coalescing (??)?

Answer: Returns right operand when left is `null` or `undefined`.

```
const name=null??"Default";    //"Default"
const value = undefined?? 42;   //42
const count = 0?? 42;           //0 (only null/undefined trigger)
```

What are private fields in classes?

Answer: Private fields (using `#`) are only accessible within the class.

```
class BankAccount {
  #balance = 0; // Private field

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}
```

Asynchronous JavaScript

What is a Promise?

Answer: A Promise represents eventual completion or failure of an asynchronous operation.

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  if (success) {
    resolve("Success!");
  } else {
    reject("Error!");
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.error(error))
  .finally(() => console.log("Done"));
```

What is async/await?

Answer: `async/await` is syntactic sugar over Promises for cleaner asynchronous code.

```
async function fetchData() {
  try {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

What is the difference between `Promise.all()` and `Promise.allSettled()` ?

Answer:

- `Promise.all()` : Rejects if any promise rejects
- `Promise.allSettled()` : Waits for all promises to settle

```
Promise.all([p1, p2, p3])
  .then(results => console.log(results))
  .catch(error => console.error(error)); // Fails fast

Promise.allSettled([p1, p2, p3])
  .then(results => console.log(results)); // Always resolves
```

What is `Promise.race()` ?

Answer: Returns the first settled promise (fulfilled or rejected).

```
Promise.race([promise1, promise2])
  .then(result => console.log(result)); // First to complete
```

What is the event loop?

Answer: The event loop handles asynchronous operations by checking the call stack and callback queue.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2 (microtasks before macrotasks)
```

What is the difference between `setTimeout()` and `setInterval()` ?

Answer:

- `setTimeout()` : Executes once after delay
- `setInterval()` : Executes repeatedly at intervals

```
setTimeout(() => console.log("Once"), 1000);  
const intervalId = setInterval(() => console.log("Repeating"), 1000);  
clearInterval(intervalId); // Stop interval
```

What are callbacks?

Answer: Callbacks are functions passed as arguments to be executed later.

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 1000);  
}  
  
fetchData((data) => {  
  console.log(data);  
});
```

What is callback hell?

Answer: Nested callbacks making code hard to read and maintain.

```
// Callback hell
getData(function(a) {
  getMoreData(a, function(b) {
    getMoreData(b, function(c) {
      // Hard to read and maintain
    });
  });
});

// Better with Promises
getData().then(a =>
getMoreData(a))
  .then(b => getMoreData(b))
  .then(c => console.log(c));
```

What is the difference between microtasks and macrotasks?

Answer:

- **Microtasks:** Promise callbacks, `queueMicrotask()` - executed before rendering
- **Macrotasks:** `setTimeout`, `setInterval`, DOM events - executed after microtasks

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
// Output: 1, 3, 2
```

How do you handle multiple async operations?

Answer:

```
// Sequential
const result1 = await
fetchData1(); const result2 = await
fetchData2();

// Parallel
const [result1, result2] = await Promise.all([
fetchData1(),
fetchData2()
]);
```

DOM Manipulation

How do you select DOM elements?

Answer:

```
// Single element
document.getElementById("myId");
document.querySelector(".myClass");

// Multiple elements
document.getElementsByClassName("myClass");
document.querySelectorAll(".myClass");
```

How do you create and add elements?

Answer:

```
const div = document.createElement("div");
div.textContent = "Hello";
div.className = "container";
document.body.appendChild(div);
```

What is event delegation?

Answer: Attaching event listener to parent to handle events from children.

```
parent.addEventListener("click", (event) => {  
  if (event.target.classList.contains("child")) {  
    // Handle click on child  
  }  
});
```

What is event bubbling and capturing?

Answer:

- **Bubbling:** Event travels from target to root
- **Capturing:** Event travels from root to target

```
element.addEventListener("click", handler, true); // Capturing  
element.addEventListener("click", handler, false); // Bubbling (default)
```

How do you prevent default behavior?

Answer:

```
element.addEventListener("click", (event) => {  
  event.preventDefault(); // Prevent default action  
  event.stopPropagation(); // Stop event bubbling  
});
```

Error Handling

What is try-catch-finally?

Answer:

```
try {
  riskyOperation();
} catch (error) {
  console.error("Error:", error.message);
} finally {
  console.log("Always executed");
}
```

How do you throw custom errors?

Answer:

```
class CustomError extends Error {
  constructor(message, code) {
    super(message);
    this.name = "CustomError";
    this.code = code;
  }
}

throw new CustomError("Something went wrong", 500);
```

What are the different error types?

Answer:

- `Error`: General error
- `TypeError`: Wrong type
- `ReferenceError`: Variable not defined
- `SyntaxError`: Syntax error
- `RangeError`: Value out of range

How do you handle errors in Promises?

Answer:

```
promise
.then(result => console.log(result))
.catch(error => console.error(error));

// With async/await
try {
const result = await promise;
} catch (error) {
console.error(error);
}
```

What is the difference between `throw` and `return` in error handling?

Answer:

- `throw`: Throws an error, stops execution
- `return`: Returns a value, stops function execution

```
function test() {
throw new Error("Error"); // Stops execution
console.log("Never executed");
}

function test2() {
return "Value"; // Returns value
console.log("Never executed");
}
```

Advanced Concepts

What is the `this` keyword?

Answer: `this` refers to the execution context of a function.

```
const obj = {
  name: "John",
  greet: function() {
    return this.name; // "John"
  },
  arrow: () => {
    return this.name; // undefined (lexical this)
  }
};
```

How does `this` work in different contexts?

Answer:

```
// Global context
console.log(this); // window (browser) or global (Node.js)

// Object method
const obj = {
  name: "John",
  greet() {
    return this.name; // "John"
  }
};

// Constructor
function Person(name) {
  this.name = name;
}
const person = new Person("John");

// Arrow function (lexical this)
const arrow = () => {
  return this; // Inherits from outer scope
};
```

What is prototypal inheritance?

Answer: Objects inherit directly from other objects through prototype chain.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.greet = function() {  
  return `Hello, I'm ${this.name}`;  
};  
  
const person = new Person("John");  
person.greet(); // "Hello, I'm John"
```

What is the prototype chain?

Answer: JavaScript looks up properties through prototype chain.

```
const arr = [];  
arr.__proto__ === Array.prototype;           // true  
Array.prototype.__proto__ === Object.prototype; // true  
Object.prototype.__proto__ === null;         // true (end of chain)
```

What is memoization?

Answer: Caching function results to avoid redundant calculations.


```
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key]) {
      return cache[key];
    }
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}
```

What is currying?

Answer: Transforming a function with multiple arguments into a sequence of functions.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function (...nextArgs) {
      return curried.apply(this, args.concat(nextArgs));
    };
  };
}

const add = curry((a, b, c) => a + b + c);
const add5 = add(5);
const add5And10 = add5(10);
console.log(add5And10(15)); // 30
```

What are generators?

Answer: Functions that can be paused and resumed using `yield`.

```
function* numberGenerator() {  
  yield 1;  yield 2;  yield 3;  
}  
  
const gen = numberGenerator();  
console.log(gen.next().value); // 1  
console.log(gen.next().value); // 2
```

What are iterators?

Answer: Objects that define iteration behavior.

```
const iterable = {  
  *[Symbol.iterator]() {  
    yield 1;  
    yield 2;  
    yield 3;  
  }  
};  
  
for (const value of iterable) {  
  console.log(value); // 1, 2, 3  
}
```

What is a Proxy?

Answer: Proxy allows intercepting and customizing operations on objects.

```
const target = {};  
const handler = {  
  get(target, prop) {  
    return prop in target ? target[prop] : 0;  
  }  
};  
  
const proxy = new Proxy(target, handler);  
proxy.name = "John";  
console.log(proxy.name); // "John"  
console.log(proxy.age); // 0 (default value)
```

What are WeakMap and WeakSet?

Answer: Collections that hold weak references (garbage collectable).

```
// WeakMap - keys must be objects  
const weakMap = new WeakMap();  
const obj = {};  
weakMap.set(obj, "value");  
weakMap.get(obj); // "value"  
  
// WeakSet - stores objects only  
const weakSet = new WeakSet();  
weakSet.add(obj);  
weakSet.has(obj); // true
```

What is debouncing?

Answer: Limiting function execution to once per time period.

```
function debounce(func, wait) {
  let timeout;
  return function(...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => func.apply(this, args), wait);
  };
}

const debouncedSearch = debounce(searchFunction, 300);
```

What is throttling?

Answer: Executing function at most once per time period.

```
function throttle(func, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      func.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

What is the difference between `var`, `let`, and `const` in loops?

Answer:

```
// var - function scoped (problematic)
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3, 3, 3
}

// let - block scoped (correct)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}
```

What is the difference between `Object.freeze()` `Object.seal()`, and `Object.preventExtensions()` ?

Answer:

```
const obj = { a: 1 };

Object.preventExtensions(obj); // Can't add properties
obj.b = 2; // Ignored

Object.seal(obj); // Can't add/remove properties
delete obj.a; // Ignored

Object.freeze(obj); // Can't modify properties
obj.a = 2; // Ignored
```

What is JSON and how do you use it?

Answer: JSON (JavaScript Object Notation) is a data format.

```
const obj = { name: "John", age: 30 };
const json = JSON.stringify(obj);      // '{"name":"John","age":30}'
const parsed = JSON.parse(json);      // { name: "John", age: 30 }
```

What is the difference between `localStorage` and `sessionStorage` ?

Answer:

- `localStorage`: Persistent, survives browser restart
- `sessionStorage`: Temporary, cleared when tab closes

```
localStorage.setItem("key", "value");
sessionStorage.setItem("key", "value");
```

What is the Fetch API?

Answer: Modern API for making HTTP requests.

```
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

What is CORS?

Answer: Cross-Origin Resource Sharing allows servers to specify which origins can access resources.

```
// Server must include CORS headers
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST
```

What is the difference between `null` and `undefined` in JSON?

Answer:

```
JSON.stringify({ a: null, b: undefined }); // '{"a":null}'
// undefined properties are omitted
```

What is the difference between `==` and `===` with examples?

Answer:

```
5 == "5"           // true (coercion)
5 === "5"          // false (strict)
null == undefined  // true
null === undefined // false
[] == false        // true (coercion)
[] === false       // false
```

What is the difference between `slice()` and `substring()` ?

Answer:

```
const str = "Hello World";
str.slice(0, 5);           // "Hello"   // "Hello"   //
str.substring(0, 5);      "World"   (negative index) //
str.slice(-5);            "Hello World" (treats as 0)
str.substring(-5);
```

What is the difference between `includes()` and `indexOf()` ?

Answer:

```
const arr = [1, 2, 3];
arr.includes(2);          // true (boolean)
arr.indexOf(2);           // 1 (index or -1)
arr.includes(5);          // false
arr.indexOf(5);           // -1
```

What is the difference between `push()` and `concat()` ?

Answer:

```
const arr1 = [1, 2];
const arr2 = [3, 4];

arr1.push(...arr2);        // [1, 2, 3, 4] (modifies arr1)
const arr3 = arr1.concat(arr2); // [1, 2, 3, 4] (new array)
```

What is the difference between `Object.assign()` and spread operator?

Answer:

```
const obj1 = { a: 1 };  
const obj2 = { b: 2 };  
  
Object.assign(obj1, obj2); // { a: 1, b: 2 } (modifies obj1)  
const obj3 = { ...obj1, ...obj2 }; // { a: 1, b: 2 } (new object)
```


Powered By
VAIBHAV SINGH



*Live a life
you remember*

<https://www.linkedin.com/in/vaibhav-singh-debugger/>

TECH
TELLIPTICA